

Multi-Cloud Architecture

Goal

Create a seamless multi-cloud deployment platform where clients can deploy Flask-React applications to any supported cloud provider by simply setting a single variable:

HOSTING_PROVIDER

Target Experience

Client Workflow (Ideal State)

In repository settings, client sets ONE variable:
HOSTING_PROVIDER: "AWS" or "DIGITAL_OCEAN" or "GCP"

- # Everything else works automatically:
- ✔ Infrastructure provisioned correctly
 - ✔ Application deploys successfully
 - ✔ Preview environments work
 - ✔ Cleanup happens automatically
 - ✔ Monitoring and alerts configured
 - ✔ SSL certificates provisioned
 - ✔ Container registry managed

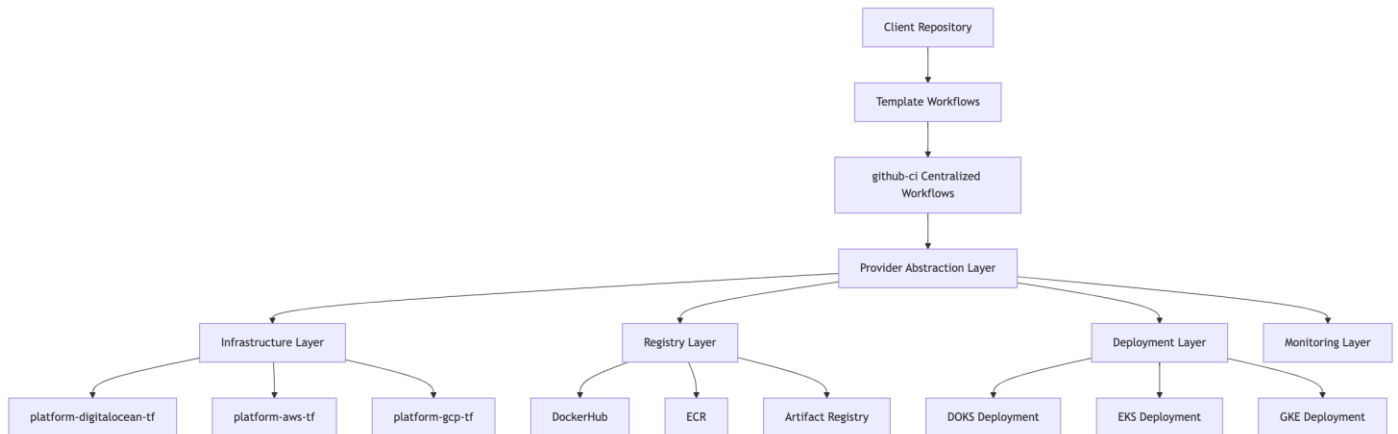
What Client Gets

Feature	DigitalOcean	AWS	GCP
Kubernetes Cluster	DOKS	EKS	GKE
Container Registry	DockerHub	ECR	Artifact Registry
Load Balancer	DO Load Balancer	ALB/NLB	Cloud Load Balancer
SSL Certificates	Let's Encrypt	Let's Encrypt	Let's Encrypt
Monitoring	DO Monitoring	CloudWatch	Cloud Monitoring
Auto-scaling	Node auto-scaling	Node group scaling	Node pool scaling
Secrets Management	Doppler	Doppler	Doppler
Preview Environments	Branch-based	Branch-based	Branch-based

Cleanup	Automatic	Automatic	Automatic
---------	-----------	-----------	-----------

Architecture Overview

Multi-Cloud Abstraction Layers



Provider Abstraction Strategy

1. Infrastructure Layer

Each provider has standardized Terraform modules

platform-digitalocean-tf/

- ├── cluster.tf # DOKS cluster with standard labels
- ├── monitoring.tf # DO monitoring setup
- └── outputs.tf # Standard outputs for CI/CD

platform-aws-tf/

- ├── eks.tf # EKS cluster with standard labels
- ├── cloudwatch.tf # CloudWatch monitoring setup
- └── outputs.tf # Standard outputs for CI/CD

platform-gcp-tf/

- ├── gke.tf # GKE cluster with standard labels
- ├── monitoring.tf # Cloud Monitoring setup
- └── outputs.tf # Standard outputs for CI/CD

2. Registry Abstraction# lib/registry/provider.sh

```
setup_registry() {
  case "$HOSTING_PROVIDER" in
    "DIGITAL_OCEAN") setup_dockerhub ;;
    "AWS")            setup_ecr ;;
    "GCP")            setup_artifact_registry ;;
  esac
}

cleanup_registry() {
  case "$HOSTING_PROVIDER" in
    "DIGITAL_OCEAN") cleanup_dockerhub ;;
    "AWS")            cleanup_ecr ;;
    "GCP")            cleanup_artifact_registry ;;
  esac
}
```

3. Deployment Abstraction

```
# lib/kube/deploy.sh

setup_node_affinity() {
  case "$HOSTING_PROVIDER" in
    "DIGITAL_OCEAN")
      export NODE_SELECTOR_KEY="doks.digitalocean.com/node-pool"
      export PRODUCTION_POOL="production-pool"
      export PREVIEW_POOL="staging-pool"
      ;;
    "AWS")
      export NODE_SELECTOR_KEY="kubernetes.githubci.com/nodegroup"
      export PRODUCTION_POOL="ng-production-pool"
      export PREVIEW_POOL="ng-preview-pool"
      ;;
    "GCP")
      export NODE_SELECTOR_KEY="kubernetes.githubci.com/nodegroup"
      export PRODUCTION_POOL="gcp-production-pool"
      export PREVIEW_POOL="gcp-preview-pool"
      ;;
  esac
}
```

Current State Analysis

What Works Today

Component	DigitalOcean	AWS	GCP
-----------	--------------	-----	-----

Infrastructure	✔ Full Support	✔ Full Support	✗ Not Implemented
Authentication	✔ Working	✔ Working	✗ Not Implemented
Build & Push	✔ DockerHub	✔ ECR Support	✗ Not Implemented
Deployment	✔ Working	✗ BROKEN	✗ Not Implemented
Cleanup	✔ Full Cleanup	✗ Missing ECR	✗ Not Implemented
Monitoring	✔ Working	✔ Working	✗ Not Implemented

Phase 1: Fix Current Multi-Cloud

Critical Bug Fixes

- **Fix Node Affinity Dynamic Selection** (Issue #87)
 - Create provider-specific node selector logic
 - Update deployment templates to use environment variables
 - Test both DigitalOcean and AWS deployments
- **Add ECR Cleanup** (Issue #88)
 - Implement ECR image deletion in clean workflow
 - Add error handling for missing images
 - Test cost optimization

Infrastructure Alignment

- **Align Terraform Labels** (Issue #3)
 - Standardize node group labels across providers
 - Add validation between Terraform and CI/CD
 - Document labeling strategy
- **Standardize Registry Patterns** (Issue #89)
 - Create registry abstraction layer
 - Simplify build action configuration
 - Implement unified cleanup patterns

Phase 2: GCP Implementation

Infrastructure Foundation

- **Create platform-gcp-tf Repository**
 - GKE cluster with standard node pools
 - Google Artifact Registry setup
 - Cloud Monitoring and alerting
 - IAM roles and service accounts

CI/CD Integration

- **Update github-ci for GCP**
 - Add GCP provider support to workflows
 - Implement Google Cloud authentication
 - Add Artifact Registry integration
 - GCP resource cleanup

Template Integration

- **Update Flask-React Template**
 - Add GCP parameters to workflows
 - Update documentation

- Add GCP setup guides

Testing & Optimization

- Comprehensive Testing
 - Three-provider testing matrix
 - Performance optimization
 - Cost analysis and optimization

Infrastructure Standardization

Standardized Node Pool Labels

Environment	Label Key	Label Value Pattern	Purpose
Production	kubernetes.githubci.com/nodegroup	{provider}-production-pool	Main workloads
Preview	kubernetes.githubci.com/nodegroup	{provider}-preview-pool	PR environments

Example Implementation:

```
# DigitalOcean
labels:
  kubernetes.githubci.com/nodegroup: "do-production-pool"
  kubernetes.githubci.com/environment: "production"
```

```
# AWS
labels:
  kubernetes.githubci.com/nodegroup: "aws-production-pool"
  kubernetes.githubci.com/environment: "production"
```

```
# GCP
labels:
  kubernetes.githubci.com/nodegroup: "gcp-production-pool"
  kubernetes.githubci.com/environment: "production"
```

Registry Strategy

Provider	Registry	Authentication	Image Pattern
DigitalOcean	DockerHub	Username/Password	docker.io/username/app:tag

AWS	ECR	AWS Credentials	{account}.dkr.ecr.{region}.amazonaws.com/app:tag
GCP	Artifact Registry	Service Account	{region}-docker.pkg.dev/{project}/repo/app:tag

Technical Implementation Details

Dynamic Configuration System

Provider Detection & Setup

```
#!/bin/bash
# lib/provider/setup.sh
setup_provider() {
    local provider=$1

    echo "Setting up for provider: $provider"

    case "$provider" in
        "DIGITAL_OCEAN")
            setup_digitalocean
            ;;
        "AWS")
            setup_aws
            ;;
        "GCP")
            setup_gcp
            ;;
        *)
            echo "ERROR: Unsupported provider: $provider"
            exit 1
            ;;
    esac
}

setup_digitalocean() {
    export REGISTRY_TYPE="dockerhub"
    export NODE_SELECTOR_KEY="doks.digitalocean.com/node-pool"
    export PRODUCTION_POOL="production-pool"
    export PREVIEW_POOL="staging-pool"
    export CLUSTER_TYPE="doks"
}

setup_aws() {
    export REGISTRY_TYPE="ecr"
    export NODE_SELECTOR_KEY="kubernetes.githubci.com/nodegroup"
```



```

export PRODUCTION_POOL="aws-production-pool"
export PREVIEW_POOL="aws-preview-pool"
export CLUSTER_TYPE="eks"
}

setup_gcp() {
  export REGISTRY_TYPE="artifact-registry"
  export NODE_SELECTOR_KEY="kubernetes.githubci.com/nodegroup"
  export PRODUCTION_POOL="gcp-production-pool"
  export PREVIEW_POOL="gcp-preview-pool"
  export CLUSTER_TYPE="gke"
}

```

Dynamic Template Generation

```

# lib/kube/templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ${KUBE_APP}
  namespace: ${KUBE_NS}
  labels:
    app: ${KUBE_APP}
    environment: ${KUBE_ENV}
    provider: ${HOSTING_PROVIDER}
spec:
  template:
    spec:
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: ${NODE_SELECTOR_KEY}
                    operator: In
                    values:
                      - ${NODE_POOL_VALUE}

```

Unified Registry Management

Registry Abstraction Interface

```

#!/bin/bash
# lib/registry/interface.sh

registry_login() {

```

```

case "$REGISTRY_TYPE" in
    "dockerhub")      dockerhub_login ;;
    "ecr")            ecr_login ;;
    "artifact-registry") artifact_registry_login ;;
esac
}

registry_push() {
    local image_name=$1
    local tag=$2

    docker push "${image_name}:${tag}"
}

registry_cleanup() {
    local image_name=$1
    local tag=$2

    case "$REGISTRY_TYPE" in
        "dockerhub")      dockerhub_cleanup "$image_name" "$tag" ;;
        "ecr")            ecr_cleanup "$image_name" "$tag" ;;
        "artifact-registry") artifact_registry_cleanup "$image_name"
"$tag" ;;
    esac
}

```

Client Configuration Matrix

Required Variables by Provider

DigitalOcean

Repository Variables

```

HOSTING_PROVIDER: "DIGITAL_OCEAN"
DO_CLUSTER_ID: "your-cluster-id"

```

Repository Secrets

```

DO_ACCESS_TOKEN: "your-do-token"
DOCKER_USERNAME: "your-dockerhub-username"
DOCKER_PASSWORD: "your-dockerhub-password"
DOPPLER_PRODUCTION_TOKEN: "your-doppler-token"
DOPPLER_PREVIEW_TOKEN: "your-doppler-preview-token"

```

AWS

Repository Variables

```
HOSTING_PROVIDER: "AWS"  
AWS_CLUSTER_NAME: "your-eks-cluster"  
AWS_REGION: "us-east-1"
```

Repository Secrets

```
AWS_ACCESS_KEY_ID: "your-aws-key"  
AWS_SECRET_ACCESS_KEY: "your-aws-secret"  
DOPPLER_PRODUCTION_TOKEN: "your-doppler-token"  
DOPPLER_PREVIEW_TOKEN: "your-doppler-preview-token"
```

GCP (Future)

Repository Variables

```
HOSTING_PROVIDER: "GCP"  
GCP_PROJECT_ID: "your-project-id"  
GCP_REGION: "us-central1"  
GCP_CLUSTER_NAME: "your-gke-cluster"
```

Repository Secrets

```
GCP_SERVICE_ACCOUNT_KEY: "base64-encoded-json-key"  
DOPPLER_PRODUCTION_TOKEN: "your-doppler-token"  
DOPPLER_PREVIEW_TOKEN: "your-doppler-preview-token"
```